Just as before, a filter can be normalized for unity gain by dividing all of the "a" coefficients by this calculated value of *G*. Calculation of Eq. 33-8 in a computer program requires a method for generating negative signs for the odd coefficients, and positive signs for the even coefficients. The most common method is to multiply each coefficient by $(-1)^k$, where *k* is the index of the coefficient being worked on. That is, as *k* runs through the values: 0, 1, 2, 3, 4, 5, 6 etc., the expression, $(-1)^k$, takes on the values: 1, -1, 1, -1, 1, -1, 1 etc.

Chebyshev-Butterworth Filter Design

A common method of designing recursive digital filters is shown by the Chebyshev-Butterworth program presented in Chapter 20. It starts with a pole-zero diagram of an *analog* filter in the s-plane, and converts it into the desired *digital* filter through several mathematical *transforms*. To reduce the complexity of the algebra, the filter is designed as a cascade of several stages, with each stage implementing one pair of poles. The recursive coefficients for each stage are then combined into the recursive coefficients for the entire filter. This is a very sophisticated and complicated algorithm; a fitting way to end this book. Here's how it works.

Loop Control

Figure 33-7 shows the program and flowchart for the method, duplicated from Chapter 20. After initialization and parameter entry, the main portion of the program is a loop that runs through each pole-pair in the filter. This loop is controlled by block 11 in the flowchart, and the FOR-NEXT loop in lines 320 & 460 of the program. For example, the loop will be executed three times for a 6 pole filter, with the loop index, P%, taking on the values 1,2,3. That is, a 6 pole filter is implemented in three stages, with two poles per stage.

Combining Coefficients

During each loop, subroutine 1000 (listed in Fig. 33-8) calculates the recursive coefficients *for that stage*. These are returned from the subroutine in the five variables: A0, A1, A2, B1, B2. In step 10 of the flowchart (lines 360-440), these coefficients are combined with the coefficients of all the previous stages, held in the arrays: A[ ] and B[ ]. At the end of the first loop, A[ ] and B[ ] hold the coefficients for stage one. At the end of the second loop, A[ ] and B[ ] hold the coefficients of the cascade of stage one and stage two. When all the loops have been completed, A[ ] and B[ ] hold the coefficients needed to implement the entire filter.

The coefficients are combined as previously outlined in Table 33-1, with a few modifications to make the code more compact. First, the index of the arrays, A[ ] and B[ ], is shifted by *two* during the loop. For example, a_0 is held in A[2], a_1 & b_1 are held in A[3] & B[3], etc. This is done to prevent the program from trying to access values outside the defined arrays. This shift is removed in block 12 (lines 480-520), such that the final recursion coefficients reside in A[ ] and B[ ] without an index offset.